

VITYARTHI - BUILD YOUR OWN PROJECT
PROJECT REPORT

FOOD SPACE

Console-Based Food Ordering & Menu Management System

STUDENT NAME

Lalit Kumar

REGISTRATION NUMBER

26BCG10012

PROGRAMME & BRANCH

B.Tech CSE (Gaming Technology)

COURSE TITLE & CODE

Python Essentials Flipped Course for CSE1021

FACULTY

Chayan Paul

TABLE OF CONTENTS

1. Introduction	7. Implementation Details
2. Problem Statement	8. Testing Approach
3. Functional Requirements	9. Challenges Faced
4. Non-Functional Requirements	10. Learnings & Key Takeaways
5. System Architecture	11. Future Enhancements
6. Design Decisions & Rationale	12. References

1. INTRODUCTION

- Manual order taking at small food counters leads to price mismatches, calculation errors, and poor daily sales tracking.
- FOOD SPACE is a lightweight, console-based Python application designed to eliminate these issues without requiring a POS terminal, database server, or internet connection.
- Customers can browse a live numbered menu, place multi-item orders, and view a running bill.
- An identity-gated path allows authorized employees to add items, remove items, or update prices, with changes saved directly to disk.

2. PROBLEM STATEMENT

Core Need: Single-counter food outlets require an accurate digital menu, error-free multi-item ordering, real-time menu management, and sales records on a single machine using only Python.

- **Scope:** Single-session console workflows for menu browsing, order modification, employee menu management, and persistent text-file storage.
- **Out of Scope:** Multi-user access, online payments, graphical interfaces, and networked operation.
- **Target Users:** Customers placing orders and employees maintaining menu accuracy.

2.1 High-Level Features

- Live menu viewing, flexible order placement, automated billing, and secure menu management.

3. FUNCTIONAL REQUIREMENTS

- **Menu Management:** Employees edit the menu, which auto-saves to `menu.txt` and updates the live numbered table via `display()`.
- **Identity & Access Control:** Employee actions require answering "Y" and verifying a `FOOD_SPACE@` ID prefix to access menu-editing options.
- **Order Placement & Modification:** Customers select items by number or name (validated against the live menu) and can modify their ongoing order.
- **Billing & Order Confirmation:** The system calculates a running total and permanently logs confirmed orders to `order_history.txt`.

4. NON-FUNCTIONAL REQUIREMENTS

- **Reliability:** The system re-prompts on invalid input to prevent crashes and auto-creates a default menu if `menu.txt` is missing.
- **Usability:** The interface uses clean console tables and allows item selection by either listed number or typed name.
- **Maintainability:** Modular functions handle specific responsibilities independently to isolate behavior changes.
- **Security:** Menu-editing actions are strictly gated behind an employee-ID prefix check.
- **Resource Efficiency:** Runs entirely on the Python standard library and two flat text files without external servers.

5. SYSTEM ARCHITECTURE

- **Interaction Layer:** Utilizes plain console I/O (`input()` / `print()`) for all customer and employee sessions.
- **Application / Logic Layer:** Comprises Identity & Access Control, Menu Management, Order Management, and Input Validation components.

6. DESIGN DECISIONS & RATIONALE

- **Flat Text Files:** Ensures zero-dependency portability so the system runs instantly upon download.
- **Flexible Item Selection:** Inputting item numbers maximizes speed, while typing names aids convenience.
- **Shared Validation:** A single `get_int()` helper standardizes numeric input checking, reducing redundant code.
- **Combined Menu Edit:** ID validation and menu editing into one readable function.
- **Prefix Check:** Utilizes a simple `FOOD_SPACE@` prefix suitable for basic offline access control.

7. IMPLEMENTATION DETAILS

- **Built in Python 3** utilizing exclusively the standard library.
- **Structured as a single script** (`food_space.py`) containing twelve single-purpose functions.
- **Data Structures:** Uses an in-memory dictionary for live menu prices/numbering and a list for accumulating orders.
- **File Formats:** `menu.txt` parses pipe-separated strings (`ITEM|PRICE`); `order_history.txt` stores append-only, human-readable receipt blocks.

8. TESTING APPROACH

- Tested boundary inputs (letters, blank spaces, negative numbers) against `get_int()` to confirm safe re-prompting.
- Verified system recovery by deleting `menu.txt` to trigger the auto-creation fallback process.

9. CHALLENGES FACED

- Building logic for menu display and updating menu.
- Dealing with `TypeError` and `ValueError` that crashed the code when you typed something different from what he was prompted.
- Creating a function-call sequence that works
- Designing text file formats that are script-parseable yet easily readable by humans without utilizing external CSV or JSON libraries.

10. LEARNINGS & KEY TAKEAWAYS

- Acquired practical experience of Python file I/O modes (read, write, append) for robust data persistence.
- Learned error handling mechanisms to deal with userinputs that raises different errors.
- Effectively using built-in data structures like dictionaries in python and applying their various methods.
- Gained useful knowledge in multi-step console workflows using functional programming and return-based control flow rather than relying purely on classes.

11. FUTURE ENHANCEMENTS

- Replacing the single python script with a multi-file Python package (`main.py`, `storage.py`, etc.) for better maintainability.
- Implement an automated testing suite using `pytest` to formally verify core functions.
- Migrate from flat text files to an SQLite database utilizing data integrity and scalability.
- Upgrade the basic prefix check to a securely hashed credential login system.
- Add daily sales summaries and support for specifying quantities (e.g., "2x SAMOSA") in a single line.
- Develop a thin web front-end (e.g., Flask) once the logic is fully modularized to replace the console interface.

12. REFERENCES

- Python Software Foundation. *Python 3 Documentation*.
- VITyarthi *Python Essentials Course*.
- Content taught in course CSE1021.